Comm/Compute Fusion on Ascend

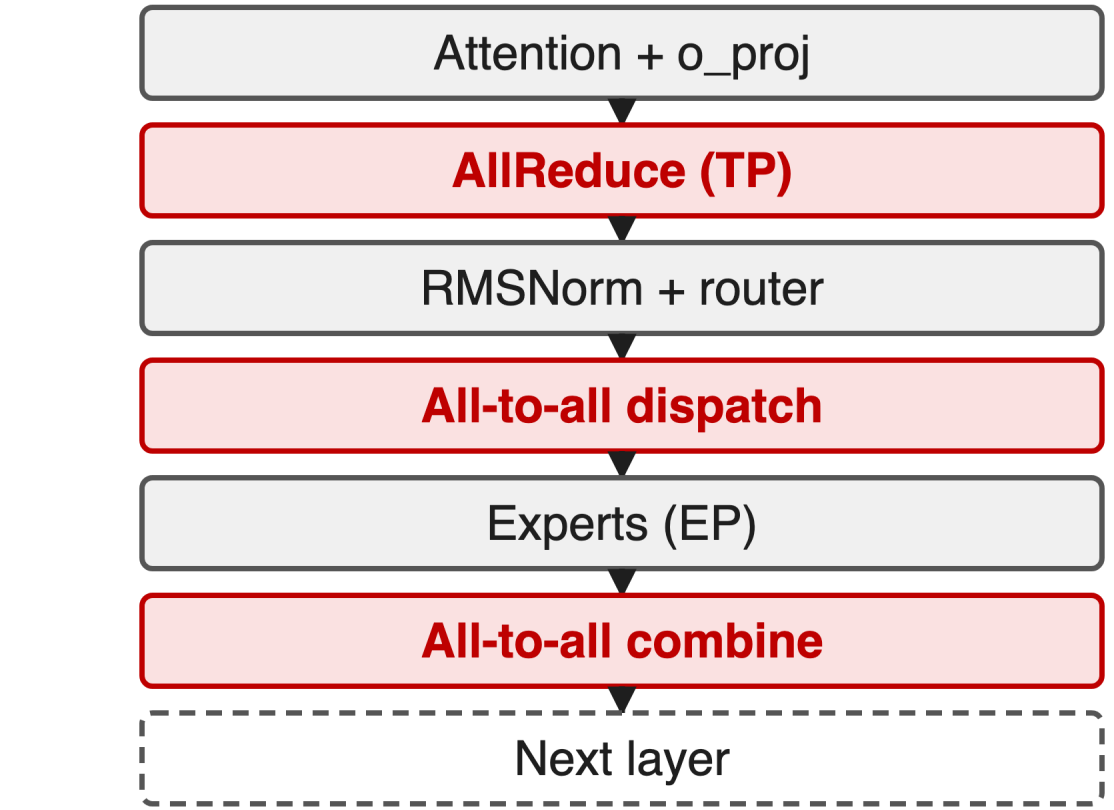
Splitting the collective, and splitting prefill from decode

Zhipu AI · GLM-5 on Ascend Atlas

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Every layer ends in a tensor-parallel collective

- GLM-5: **744B** MoE, **40B** active, **200K** context
- One Atlas 800 A3 node, TP up to **16** inside it
- Attention all-reduces; the MoE all-to-alls



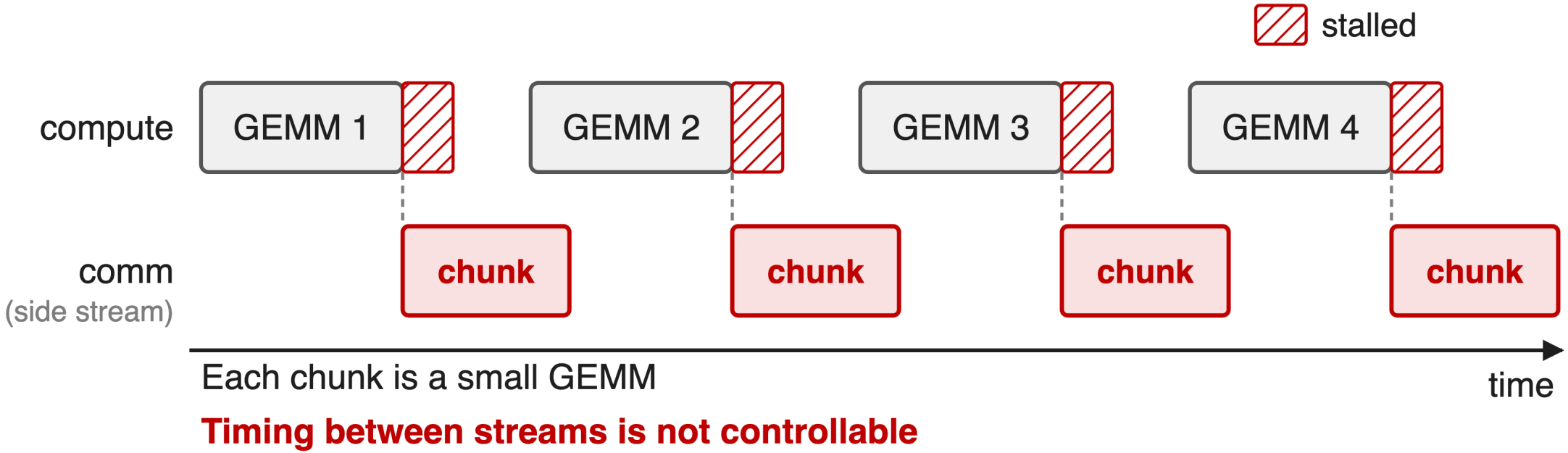
Weak links push TP communication to 40–75%

40–75%

of step time is tensor-parallel communication on an A100 PCIe cluster — only 8–11% on NVLink

Flux, arXiv 2406.06858, §6 p.15 — GPU clusters, not Ascend; no Ascend percentage is published

Chunked overlap can be slower than none



Measured overlap efficiency: –125% to 36% on the weak-fabric cluster

Insight: AllReduce is ReduceScatter plus AllGather

- Each half now borders a matmul it can ride along with

Insight: AllReduce is ReduceScatter plus AllGather

- Each half now borders a matmul it can ride along with

And between the halves every rank holds only 1/TP of the tokens

Hide it at three scales

❶ Layer

Split the collective

AllReduce → RS + AG

Norm and router run on sharded tokens

❷ Kernel

Fuse it into the matmul



Every tile ships its partial inside one kernel

❸ Cluster

Split the roles

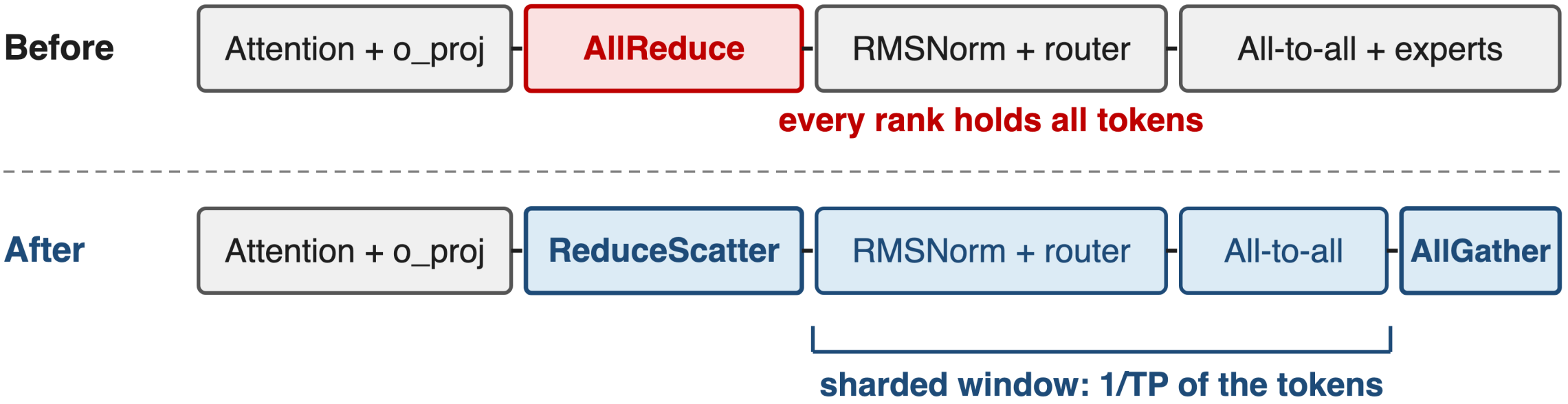
Prefill nodes → Decode nodes

Prefill cannot preempt a running decode

Same goal at three scales: no rank waiting on the fabric

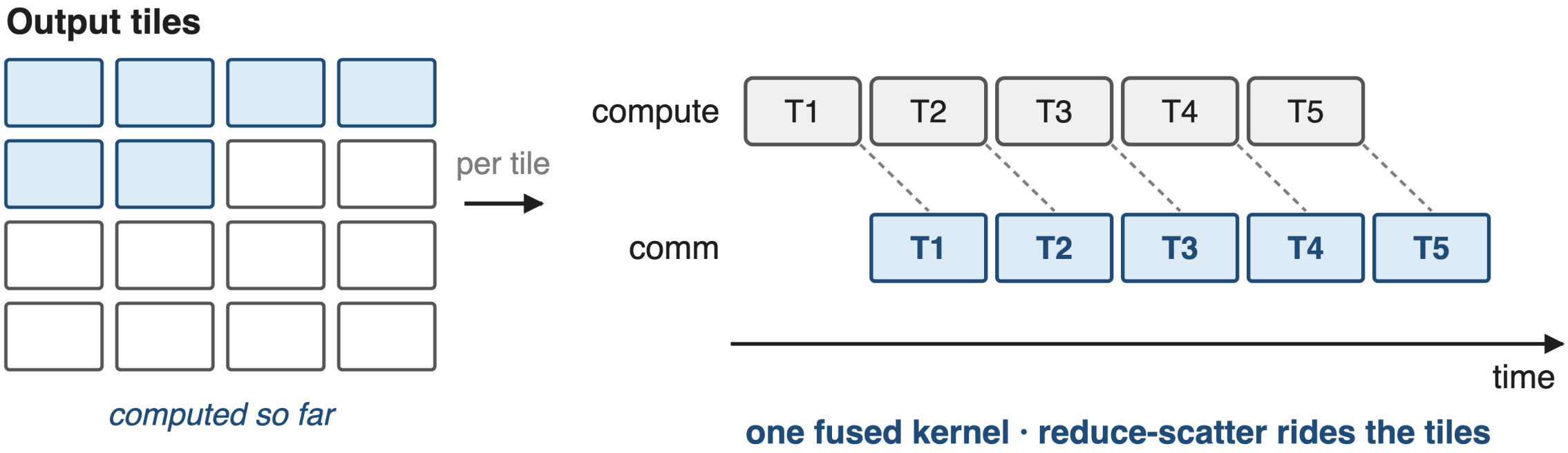
❶ FlashComm split · ❷ fused matmul and reduce-scatter · ❸ prefill/decode disaggregation

❶ The split leaves a sharded window



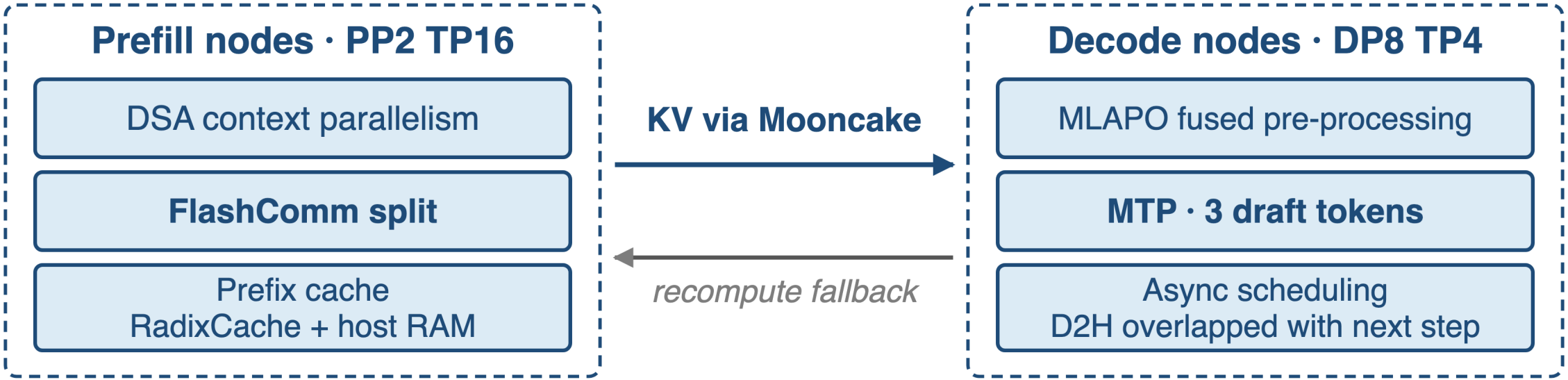
RMSNorm, router and all-to-all then see 1/TP of the tokens

❷ Each matmul tile ships its own partial



Upstream: `fuse_gemm_comms` fuses a row-parallel projection with its reduce-scatter (BF16/FP16, TP 2/4/8)

❸ Prefill and decode get their own nodes



Long prefills can no longer preempt running decodes

one Atlas 800 A3 node each

vLLM Ascend reference configuration for 198K contexts; Mooncake carries the KV cache

MTP drafts land 2.76 tokens per step

- GLM-5 shares **3** MTP layers in training, at one layer's memory cost
- Decode nodes speculate **3** tokens; prefill nodes **1**

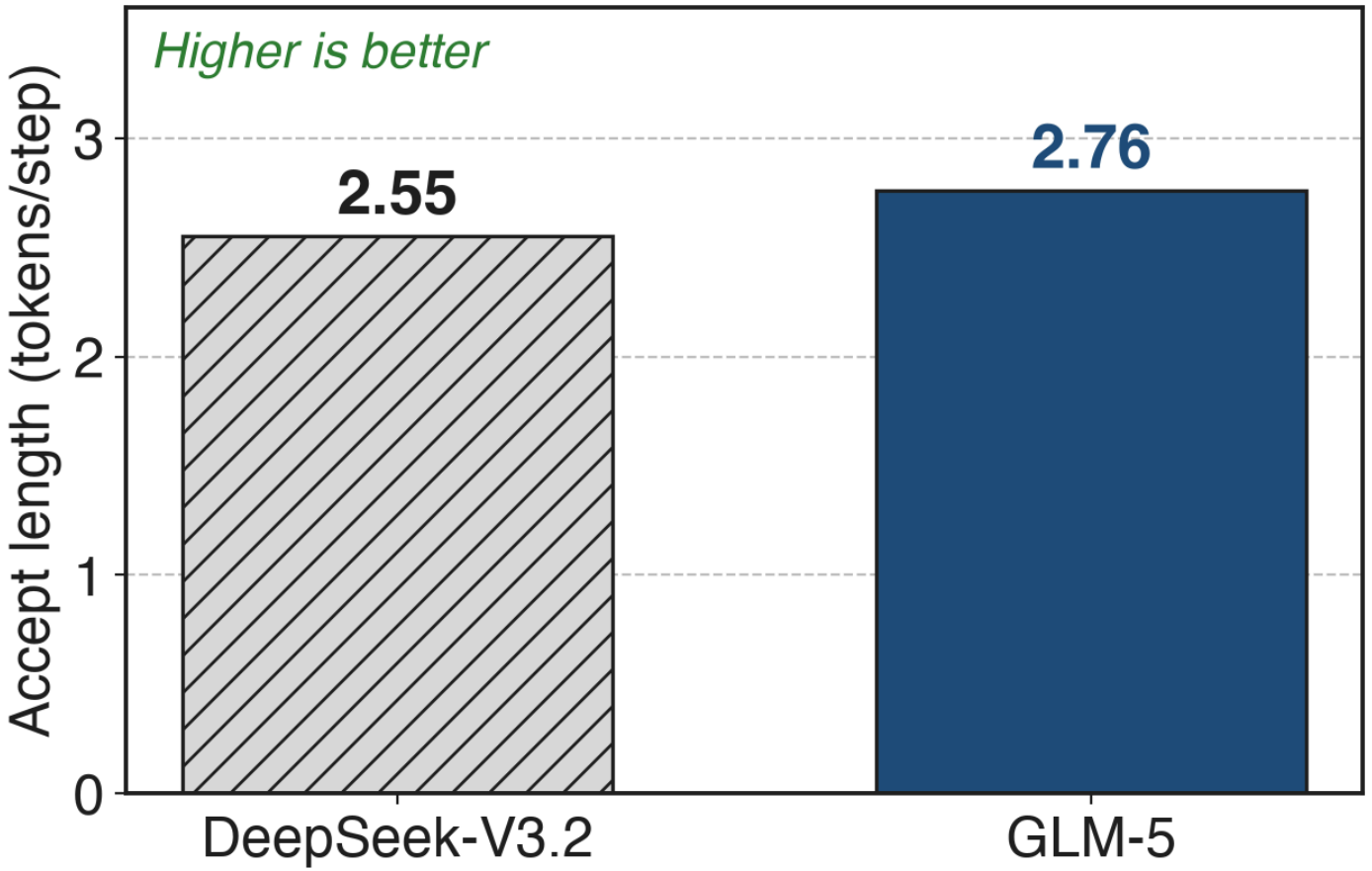


Table 2: 4 speculative steps, the authors' private prompt set

The report claims one node, half the cost

- One domestic node "comparable to dual-GPU international clusters"
- **50%** lower deployment cost in long-sequence scenarios
- **No per-feature number** for the split, the fusion or the PD path

That 50% covers quantization, fused kernels and scheduling together

The fine print on every switch

- FlashComm: mainly **prefill**; not with `o_proj` sharding
- Fused MoE ops: **w8a8**, EP ≤ **32**, no MTP
- Fused matmul + RS: BF16/FP16, **TP 2/4/8**
- PD: extra nodes, a KV path, a recompute fallback

All of them must be switched on explicitly

Summary

- TP communication: **8–11%** on NVLink, **40–75%** on a weak fabric (Flux)
- **Insight**: AllReduce = ReduceScatter + AllGather, and the gap is sharded
- Split the collective · fuse it into the matmul · split prefill from decode
- GLM-5 claims one node and **50%** lower long-sequence cost, for the whole stack

https://arxiv.org/abs/2602.15763